Trust-Weighted Gossip for Decentralized Storage and Retrieval

A Protocol for Returning Information Custody to the Social Graph

Version 1.3 — 2026-07

Leon Acosta
leon@gozzip.org

July 2026

Abstract

Decentralized social protocols—Nostr, ActivityPub (Mastodon), AT Protocol (Bluesky)—still depend on servers that control what gets stored, served, and censored. This paper presents an open, censorship-resistant protocol for social media and messaging that returns data custody to the social graph itself. The protocol inherits Nostr's proven primitives—secp256k1 identity, signed events, relay transport—and adds a storage and retrieval layer where users own their data. Because events are self-authenticating (signed by the author's keys), public content (posts, reactions, reposts) can be exported to ActivityPub, AT Protocol, and RSS/Atom via bridge services; protocol-specific features (pacts, WoT routing, device delegation, encrypted DMs) are not bridgeable. Users form reciprocal *storage pacts* with trust-weighted peers, creating a distributed storage mesh that mirrors how human communities naturally preserve and transmit information. All protocol intelligence—gossip forwarding, encrypted request routing, WoT filtering, device resolution—lives in clients. Standard Nostr relays work without any modifications; optimized relays can optionally accelerate specific operations. We describe the pact formation mechanism, a three-tier retrieval protocol with cascading fallback paths, and a Web of Trust (WoT)-filtered gossip layer that bounds propagation while achieving high-probability delivery within the trust boundary. We further describe how the protocol builds on the iroh peer-to-peer stack (QUIC transport, Ed25519 endpoint identity, relay-assisted hole punching) for direct node-to-node connectivity, with mesh radio, Bluetooth, and Tor multipath as future extensions that reduce dependence on the internet itself.

1 Introduction

1.1 The Gossip Parallel

Human communities have always propagated information through gossip. A person shares news with their close circle, who share it with theirs, creating epidemic spread through a trust-weighted social graph. This mechanism has three properties that formal gossip protocols seek to replicate:

1. **Trust filtering**—information from a close friend carries more weight than from a stranger. A claim about person X means different things coming from someone 1 hop versus 4 hops away.
2. **Contextual preservation**—gossip within a community preserves context: who said what, to whom, under what circumstances. Gossip without context is slander; with context it is signal.

3. **Natural redundancy**—important information reaches you through multiple independent paths. If one friend is unavailable, another provides the same update. The redundancy is proportional to the information's social relevance.

These properties map directly onto the protocol primitives we describe: WoT-filtered forwarding (Section 4), capped reciprocal storage pacts (Section 5), and multi-path retrieval with cascading fallback (Section 6).

1.2 The Relay Problem

Decentralized social protocols promise user sovereignty, but all of them depend on servers that control what gets stored, served, and censored. In Nostr [1], relays decide which events to keep and for how long. In Mastodon/ActivityPub [12], your account is bound to an instance operator who can suspend it, shut down, or change their terms. In Bluesky's AT Protocol, a centralized relay aggregates and serves the firehose. The specific architecture varies, but the dependency is the same:

- **Storage control**—servers decide which data to store and for how long.
- **Distribution control**—servers decide which data to serve and to whom.
- **Censorship capability**—operators can silently drop content or suspend accounts.
- **Economic leverage**—users must pay operators or accept their terms.

This recreates the platform dependency that decentralized protocols were designed to eliminate. The user's social graph lives on infrastructure they do not control.

1.3 Contribution

We propose a local-first architecture where:

- **Primary storage** resides on the user's device and their close WoT peers (1–2 hops).
- **Primary retrieval** queries the trust network first, not relay infrastructure.
- **Relay role** shifts from data custodian to delivery infrastructure with reduced custody—still structurally important for bootstrap, discovery beyond the WoT, mobile-to-mobile mailbox, and push delivery, but no longer the canonical storage layer. This is reduced relay custody, not optional infrastructure.
- **Data is self-authenticating and portable.** Every event is signed by the author's keys. Public content (posts, reactions, reposts) can be exported to ActivityPub, AT Protocol, and RSS/Atom via bridge services. Protocol-specific features (pacts, WoT routing, device delegation, encrypted DMs) are not bridgeable. Cross-protocol identity linking uses signed attestations.

This is gossip protocol in the computer science sense—epidemic information spreading through a peer mesh [2]—with WoT determining propagation priority. The protocol inherits Nostr's proven primitives—secp256k1 keypairs, signed events, relay transport—and works with existing Nostr relays and clients from day one. All protocol intelligence lives in clients; standard relays store and forward events without modification. The combination of Nostr's key model, social-graph storage, and trust-weighted routing creates a system where the network's social structure *is* its infrastructure.

The paper makes three contributions:

1. A **storage pact mechanism** with capped asymmetric reciprocity, challenge-response availability verification, and popularity-scaled redundancy (Section 5).
2. A **three-tier retrieval protocol** with cascading delivery paths—local pact storage, direct partner query, and relay fallback (Section 6), with WoT gossip as an optional censorship-resistance extension.
3. An **iroh transport architecture** that gives each node direct QUIC connectivity with Ed25519 endpoint identity and relay-assisted hole punching, extensible to mesh radio, Bluetooth, and Tor multipath (Section 8).

2 Communities and Protocols: A Structural Parallel

The design of this protocol draws inspiration from human social dynamics. Each protocol primitive maps to an observable pattern in how humans form communities, maintain relationships, and propagate information.

2.1 Inner Circle, Outer Circle, Acquaintances

Human social networks exhibit concentric structure. Robin Dunbar's research [3] identifies layers: ∼5 intimate contacts, ∼15 close friends, ∼50 good friends, ∼150 casual friends, with each layer roughly tripling in size and halving in intimacy.

The protocol mirrors this with three node roles:

Human Layer	**Protocol Equivalent**	**Persona**	**Storage Obligation**	**Uptime**
Inner circle (5–15)	Full nodes (5–25%)	*Keeper*	Complete event history	95%
Extended circle (50–150)	Light nodes (75%)	*Witness*	Rolling 30-day window	30% (Phase-2 mobile)
Acquaintances (150+)	Relay-discovered peers	*Herald*	None (optional caching)	Variable

Full nodes are the protocol's equivalent of the friends who remember everything—your complete history is safe with them. Light nodes are the broader social circle: they know what you've been up to recently, but don't maintain archives. Acquaintances discovered through relays are the weak ties [4] that provide reach beyond your community, without storage commitment.

2.2 Reciprocity as Infrastructure

In human communities, relationships survive through reciprocity. You remember my stories; I remember yours. One-sided relationships decay. The protocol formalizes this as *storage pacts*: bilateral agreements where both parties store each other's events.

Reciprocity here does not require equal output. Rather than metering friendship by matching data volumes, each partner simply stores whatever the other produces, up to a modest per-partner cap (∼10 MB/month). This *capped asymmetric* model is the protocol equivalent of a friendship that does not keep score: a prolific poster and a quiet lurker can hold a pact without either bearing runaway cost, and the 60–80% of users who mostly read are readmitted to the storage economy instead of being priced out by a volume-balance requirement.

2.3 Reputation Through Behavior, Not Scores

Human reputation is not a number. It is the aggregate of observed behavior, weighted by the observer's trust in each source. The protocol's reliability scoring operates identically:

- Each node maintains **per-peer reliability scores** using a rolling ∼3-week window (≈20 samples) of challenge-response results.
- Scoring is **presence-aware**: a partner is judged only on challenges issued while it was observed online, so an intermittently-online *Witness* is not marked a defector merely for sleeping.
- Scores are private—each node computes its own assessment.
- Four bands govern action: Healthy ($\geq$ 90%), Degraded (70–90%, challenge more often and begin seeking a replacement), Unreliable (50–70%, actively replacing), and Failed ($<$ 50%). A partner is dropped only when it stays in Failed.
- Failed peers naturally lose their reciprocal storage—the same way unreliable friends are gradually excluded from information flow.

There is no global reputation score. The network topology *is* the incentive structure. A node with 20 reliable pact partners has 20 advocates forwarding its content through gossip. A dropped pact means a lost advocate and reduced reach—the protocol equivalent of being talked about less.

2.4 Guardianship—Bootstrapping Through Generosity

Every community has patrons—established members who vouch for newcomers they don't personally know. A shopkeeper who gives a stranger their first job. A neighbor who co-signs a lease for a new arrival. These acts of generosity bootstrap trust where none exists.

The protocol formalizes this as *guardian pacts*: an established user (*Guardian*) voluntarily stores data for one untrusted newcomer (*Seedling*) outside their Web of Trust. Unlike bootstrap pacts (triggered by a follow), guardian pacts are volunteered—the Guardian opts in, accepting a small storage cost to help the network grow.

	Bootstrap Pact	**Guardian Pact**
Trigger	Seedling follows someone	Guardian volunteers
WoT required	No (follow creates edge)	No
Capacity	One per followed user	One per Guardian
Expiry	90 days or 10 reciprocal pacts	90 days or Hybrid phase (5+ pacts)
Reciprocity	One-sided	One-sided

The pay-it-forward framing is deliberate: today's Seedling becomes tomorrow's Guardian. A user who was supported during their bootstrap phase is encouraged (by client UX, not protocol enforcement) to volunteer a guardian slot once they reach Sovereign phase.

Incentive limitation: Guardian pacts are one-sided—the Guardian stores data for a Seedling and receives nothing in return. Game-theoretic analysis shows the unique Nash equilibrium is "nobody volunteers"—a classic public goods problem where individual rationality produces collective failure. The pay-it-forward framing relies on prosocial behavior, not rational self-interest. Operational deployment should include either: (a) Genesis Guardian nodes (5–10 operator-run full nodes providing guardian services during the first 6–12 months), (b) a guardian incentive mechanism (gossip priority boost, persistent WoT edge after successful guardianship), or (c) both. Guardian pact design is an active area of development.

2.5 Gossip as Curated Propagation

When humans gossip, they don't broadcast to everyone. They share selectively based on trust and relevance. The protocol's WoT-filtered forwarding mirrors this:

- **Active pact partners** (highest priority)—you forward eagerly for those whose data you store.
- **1-hop WoT peers**—people you directly follow. You'll pass along their content.
- **2-hop WoT peers**—friends-of-friends. You'll forward if capacity permits.
- **Unknown sources**—never forwarded. Strangers' gossip stops at your door.

This creates a natural boundary for information propagation that matches Dunbar's social brain hypothesis [3]: the protocol's gossip radius is bounded by the same trust gradient that bounds human social information flow.

3 Network-Theoretic Foundations

The protocol's design is grounded in results from network science. This section maps each key design decision to the formal theory that justifies it, establishing why the social graph is a viable substrate for decentralized infrastructure.

3.1 Small-World Property and Gossip Reach

Watts and Strogatz [13] demonstrated that networks with high clustering and short path lengths—*small-world* networks—emerge from even minor random rewiring of regular lattices. In the small-world regime, characteristic path length scales as $L \sim \ln(N)/\ln(k)$ while the clustering coefficient C remains close to its lattice value ($\sim 3/4$). Social networks consistently exhibit this structure: densely clustered local neighborhoods connected by a few long-range shortcuts.

This directly justifies the protocol's gossip TTL of 3 hops. In a small-world network of N nodes with mean degree k, the expected number of nodes reachable within h hops (accounting for clustering coefficient C) is:

$$\text{reach}(h) \approx k \cdot [k(1-C)]^{h-1} \tag{1}$$

For a 5,000-node network with $k = 20$ and $C = 0.25$: reach(1) = 20, reach(2) = 300, reach(3) $\approx$ 4,500 (up to $20^3 \approx 8{,}000$ under a naive no-overlap count, $\approx$4,500 once clustering is deducted). Three hops theoretically cover 90%+ of the network. This analytical reach is an upper bound, not a delivery rate: gossip is a secondary, optional path in the retrieval cascade (Section 6), and the protocol's effectiveness derives primarily from pact-local reads rather than gossip propagation, with relay fallback (Tier 3) handling content beyond the pact set. Increasing TTL to 4 would add minimal reach at substantial bandwidth cost—the logarithmic path length means most information arrives within 2–3 hops.

The small-world structure also explains why WoT-bounded gossip works at all: the combination of local clustering (friends share friends) and short paths (any two nodes are connected by a few hops) means that a gossip message restricted to 2-hop WoT peers can still reach most of the relevant network through trust-weighted forwarding.

3.2 Scale-Free Topology: Robustness and Vulnerability

The protocol's simulation uses Barabási–Albert (BA) preferential attachment [14] to generate social graphs with power-law degree distributions $P(k) \sim k^{-\gamma}$. This choice follows a common

modeling convention: online social networks exhibit heavy-tailed degree distributions with properties qualitatively similar to scale-free networks, though whether they follow strict power-law distributions is debated [15]. The BA model is a useful approximation—not an exact match—for exploring the protocol's behavior on heterogeneous graphs. The simulator also supports Watts–Strogatz small-world graphs [13] and LFR benchmark graphs [25] for community structure analysis.

Graph density and protocol performance: Graph structure is expected to shape protocol performance in ways worth testing. Sparser graphs with more pronounced hub structure should improve pact partner diversity and reduce correlated failures relative to dense graphs, while small-world topologies should rely more on gossip forwarding to compensate for weaker hubs. Quantifying these effects is part of the pending empirical validation against the rebuilt simulator.

Albert, Jeong, and Barabási [16] proved a fundamental asymmetry in scale-free networks:

- **Random failure**: Networks with heavy-tailed degree distributions tolerate significant random node removal before losing their giant component. Low-degree nodes (the vast majority) contribute little to global connectivity, so their failure has negligible effect. The exact tolerance depends on the degree distribution; for networks with finite second moment (log-normal or truncated power-law, common in real social networks), the percolation threshold is non-trivial but still high.
- **Targeted attack**: Removing nodes in order of decreasing degree fragments scale-free networks after removing as few as 5–18% of the highest-degree hubs.

Cohen et al. [17] formalized this using percolation theory. The giant component survives random removal as long as $\kappa = \langle k^2 \rangle / \langle k \rangle > 2$ (the Molloy–Reed criterion). For idealized scale-free networks with $\gamma \leq 3$, the second moment diverges and the percolation threshold approaches 1. Real social networks have finite degree distributions, so κ is finite—but the protocol's target of 20 active pacts (floor 12) provides local protection regardless of the global degree distribution.

These results directly inform three protocol design decisions:

1. **Pact redundancy (target 20 active, floor 12)**: Random node failures (mobile devices going offline, churn) are the dominant failure mode. With ∼20 pact partners, the protocol exploits scale-free robustness—data survives because the loss of any individual partner is statistically insignificant.
2. **Eclipse attack defense**: The targeted-attack vulnerability motivates the WoT-only pact formation rule. An attacker must first infiltrate the target's WoT (mutual follows) before offering pacts—a social barrier that prevents the degree-based targeting that would be devastating in an open network.
3. **Renewal-by-default replacement**: Healthy pacts auto-renew, and a partner that sustains a Failed reliability band is replaced by forming a fresh pact through the diversity heuristic. This addresses the narrow window between targeted hub removal and network fragmentation: continuous replacement maintains connectivity during recovery rather than relying on a pre-provisioned standby reserve.

3.3 Epidemic Spreading and the WoT Boundary

Pastor-Satorras and Vespignani [18] proved that the epidemic threshold for spreading processes on scale-free networks vanishes:

$$\lambda_c = \frac{\langle k \rangle}{\langle k^2 \rangle} \to 0 \quad (\text{for } \gamma \leq 3) \tag{2}$$

This means any non-zero transmission rate produces epidemic spreading across the entire network. For gossip protocols, this is a double-edged result: gossip *will* spread efficiently (good for retrieval), but so will spam, misinformation, and attack traffic (bad for security).

The protocol resolves this tension through two complementary mechanisms: a *social* restriction (WoT-only forwarding) and a *structural* restriction (TTL=3 hop limit). Together, these restore a non-zero epidemic threshold on a network that would otherwise have none.

The vanishing threshold above is precisely what the WoT boundary defends against. On the unrestricted scale-free graph, any gossip message—including spam and attack traffic—would reach epidemic prevalence regardless of its transmission rate. The WoT restriction changes the effective propagation topology: a node forwards gossip only to peers within its Web of Trust, so the relevant adjacency matrix is not the full social graph but the *realized WoT subgraph* induced by mutual-follow and trust-scored relationships. Castellano and Pastor-Satorras [19] showed that the epidemic threshold is governed by the spectral radius λ_1 of this effective adjacency matrix ($\lambda_c = 1/\lambda_1$). Because WoT-restricted forwarding removes the long-range shortcuts that hub nodes would otherwise provide to arbitrary traffic, the spectral radius of the WoT subgraph is substantially smaller than that of the full network, yielding a finite, non-trivial threshold.

Crucially, this threshold depends on the *global* structure of the realized WoT topology—not on a local degree cap. Hub nodes within any given node's 2-hop neighborhood may still have high degree, but they will only forward content that originates from or is endorsed by their own WoT. The TTL=3 hop limit provides an independent bound: regardless of the WoT subgraph's spectral properties, no single gossip message can propagate beyond three forwarding hops, which caps the reachable set for any originating node and prevents runaway cascades even in densely connected WoT clusters.

This creates a **dual regime**: within the WoT boundary, retrieval stays inside the trust network (local storage and direct partner queries, with optional gossip); beyond the boundary, propagation requires relay assistance. The protocol's three-tier retrieval cascade (Section 6) is the operational implementation of this theoretical boundary: Tiers 1–2 keep reads intra-WoT, while Tier 3 (relay fallback) handles the inter-community gap.

3.4 Triadic Closure and 2-Hop WoT Effectiveness

Rapoport [20] first formalized triadic closure: if A is connected to both B and C, the probability that B and C become connected is significantly higher than random. This structural tendency drives the clustering coefficient in social networks and is the mechanism that makes 2-hop WoT meaningful.

The protocol's 2-hop tier exploits triadic closure directly. If Alice (node A) mutually follows Bob (node B), and Bob follows Carol (node C), then:

- The probability that Carol is relevant to Alice is proportional to the number of Alice's mutual contacts who follow Carol (the trust score).
- A Carol followed by 8 of Alice's 15 mutual contacts is far more likely to be a genuine community member than one followed by 1.

The trust score in the 2-hop WoT map is precisely this count of closing triads. Granovetter's weak ties theory [4] complements this: the 1-hop tier (non-mutual follows) represents weak ties that bridge communities, while the direct WoT tier (mutual follows) represents strong ties with high triadic closure. The 50/30/20 weight split reflects Granovetter's finding that strong ties provide redundancy (high overlap in information) while weak ties provide novelty (access to distant communities).

3.5 Community Structure and Information Boundaries

Girvan and Newman [21] demonstrated that real networks have modular community structure detectable through edge betweenness. Edges connecting communities carry disproportionately many shortest paths—they are information bottlenecks.

The modularity metric Q quantifies this structure:

$$Q = \sum_c \left[\frac{L_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right] \tag{3}$$

where L_c is edges within community c, m is total edges, and d_c is the total degree of community c. Values of $Q > 0.3$ indicate significant community structure.

This has two implications for the protocol:

1. **Gossip confinement**: In high-modularity networks (typical of social graphs), gossip propagating within a WoT community rarely crosses community boundaries because few edges span the gap. This is a *feature*: read requests for a community member's content are served by community members, keeping traffic local. The inter-community relay fallback handles the exceptions.

2. **Privacy through topology**: Reader privacy in this protocol comes from *where* request metadata flows—to a handful of WoT peers rather than to arbitrary relay operators (Section 6)—and community structure reinforces this. A retrieval request for Alice's data stays within Alice's community; nodes outside the community never see it, an additional topological layer on top of the confidentiality provided by encrypting the request itself.

3.6 Redundant Paths and Fault Tolerance

Menger's theorem [22] states that the maximum number of vertex-disjoint paths between two nodes equals the minimum vertex cut separating them. For a k-vertex-connected graph, any node can tolerate the failure of $k - 1$ neighbors without losing connectivity to any other node.

Applied to the pact topology: if a user maintains 20 active pact partners, and each partner is connected to the user through the WoT, then Menger's theorem guarantees that an adversary must compromise at least as many nodes as the minimum vertex cut to isolate the user's data. In practice, the WoT's high clustering means multiple independent paths exist between pact partners, providing redundancy beyond what the raw pact count suggests.

The diversity heuristic in pact formation adds a layer: partners are chosen to span different WoT communities, so even after min-cut nodes are compromised, replacement pacts provide fallback paths that may traverse entirely different parts of the graph.

3.7 Dunbar Layers as Protocol Parameters

Dunbar's research [3] identifies fractal social layers: ∼5 intimate contacts, ∼15 close friends, ∼50 good friends, ∼150 casual friends, each layer roughly 3× the previous. The protocol's parameters are calibrated to these layers:

The 20-pact target sits within the sympathy group layer (∼15), reflecting that storage pacts require ongoing reciprocal obligation—a level of commitment compatible with the ∼15 relationships humans maintain with regular contact. The 2-hop WoT boundary (∼150 effective peers) aligns with Dunbar's number, beyond which trust assessment becomes unreliable.

The design draws inspiration from Dunbar's social layer model (5/15/50/150), though the protocol's parameters are ultimately justified by the availability and storage calculations rather

Dunbar Layer	Size	Protocol Mapping	Parameter
Support clique	∼5	Full-node pact partners	∼5 Full pacts (25% of 20)
Sympathy group	∼15	Active pact partners	20 active pacts
Affinity group	∼50	Direct WoT tier	Mutual follows
Dunbar number	∼150	1-hop + 2-hop WoT tiers	Follow list + extended network
Acquaintances	∼500	Relay-discovered peers	Beyond WoT boundary

than by strict social-layer correspondence. Online social networks may diverge from Dunbar's predictions in important ways (asymmetric follows, larger follow lists, parasocial relationships). Nevertheless, the broad intuition holds: a protocol that required 500 pact partners would exceed most users' reciprocal relationship capacity, and one that relied on 5-hop WoT would extend trust beyond meaningful social distance.

4 Protocol Primitives

4.1 Identity and Trust

Each participant holds a secp256k1 keypair, compatible with Nostr's identity model [1]. The public key serves as both identity and address. A key hierarchy isolates device-level operations from identity-level authority:

- **Root key** (cold storage)—signs device delegations only.
- **Governance key**—signs profile and follow list updates.
- **Device subkeys**—sign day-to-day events (posts, reactions, DMs).

Subkeys are derived using HKDF-SHA256 (RFC 5869) with IKM = root private key (32 bytes), salt = `gozzip-v1` (protocol constant), info = purpose label string (e.g., `governance`, `device/0`), output = 32 bytes interpreted as a secp256k1 scalar (reduced mod curve order n; retry with incremented counter if zero).

The Web of Trust is defined by follow relationships. A node p follows a set of nodes F_p. The WoT distance $d(p, q)$ is the minimum number of follow-hops from p to q. The protocol operates within a 2-hop boundary: nodes forward gossip only to peers where $d(p, q) \leq 2$.

4.2 Node Types

Let $N = \{n_1, n_2, \ldots, n_k\}$ be the set of network participants. Each node n_i has type $t_i \in \{\text{Full}, \text{Light}\}$:

- **Full nodes** (*Keepers*) maintain complete event history for their pact partners. Expected uptime: $u_{\text{full}} = 0.95$.
- **Light nodes** (*Witnesses*) maintain events within a checkpoint window W (default 30 days) for their pact partners. Expected uptime: $u_{\text{light}} = 0.30$ (a Phase-2 mobile assumption; true background uptime is lower, 0.3–5%).

The 30% light node uptime figure assumes the user actively opens the app several times daily. Mobile OS background execution constraints (iOS BGAppRefreshTask, Android Doze) limit true background uptime to 0.3–5%. The protocol's availability calculations remain valid at these lower figures because full nodes (Keepers) dominate the availability guarantee; even if all light nodes have near-zero uptime, the full-node pact partners provide the critical redundancy.

The protocol is *intended* to degrade gracefully down to full-node ratios as low as ∼5%. The 25% ratio used in the baseline analysis represents an optimistic target; comparable decentralized systems achieve 0.1–5% always-on node participation. The 5% regime is analytically plausible under independence assumptions (the all-light-node analysis gives $P(\text{unavailable}) = 0.08\%$) but has *not* been simulated, and at 5% the per-full-node pact load (∼400 pacts, i.e. 20/0.05) would exceed the ∼200-pact comfort threshold flagged elsewhere. The baseline analysis uses approximately 25% Full nodes (always-on servers, dedicated hardware) and 75% Light nodes (mobile devices, intermittent connectivity).

4.3 Events

An event $e = (\text{id}, \text{author}, \text{kind}, \text{size}, \text{seq}, \text{prev\_hash}, \text{created\_at})$ is the atomic unit of information. Events are cryptographically signed by the author's device key and are immutable once published.

The weighted average event size under the default content mix is:

Formula F-01:

$$\mathbb{E}[\text{size}] = \sum_k \text{size}_k \cdot \text{mix}_k = 800(0.40) + 500(0.30) + 600(0.15) + 900(0.10) + 5500(0.05) = 925 \text{ bytes} \tag{4}$$

This calculation covers text events only. Media (images, video, audio) is stored separately via content-addressed hash references in `media` tags, keeping event sizes small regardless of media attachment count. See Section 5 (Media References) for details.

4.4 Checkpoints

Checkpoints (kind 10051) are periodic reconciliation markers published by each user. A checkpoint contains:

- Per-device event heads (latest event ID and sequence number per device).
- A Merkle root over all events since the previous checkpoint.
- References to current profile and follow list.

Checkpoints enable light nodes to verify data completeness without downloading full history, and define the storage obligation boundary for light pact partners.

Merkle tree construction: The checkpoint Merkle root is a SHA-256 binary hash tree. Leaves are the 32-byte event IDs (the SHA-256 hash that Nostr already computes for each event), ordered canonically by `(device_pubkey, seq)`—device pubkey lexicographic first, then sequence number ascending. If the number of leaves is not a power of two, the last leaf is duplicated to fill the next power of two. Internal nodes are computed as $H(\text{left\_child} \| \text{right\_child})$. Two implementations given the same event set MUST produce the same root.

Protocol versioning: All custom event kinds (10050–10064) include a `protocol_version` tag (currently `"1"`). Clients receiving events with an unknown version process them on a best-effort basis. Breaking changes increment the version number. Pact partners verify version compatibility during formation; incompatible versions trigger pact replacement.

4.5 Media References

Events that include media (images, video, audio) carry content-addressed hash references rather than inline data. Each media file is stored externally—on CDNs, S3, IPFS, or self-hosted

servers—and referenced via a `media` tag containing the SHA-256 hash of the raw bytes, MIME type, byte count, and a URL hint. The event itself remains ∼1 KB regardless of the number of media attachments.

Media volume is excluded from event pact obligations. A peer-to-peer media redundancy layer (*media pacts*) is *deferred* and not part of protocol v1; the content-addressed reference described here is the piece that ships, and clients verify SHA-256(downloaded_bytes) = hash_from_media_tag on every fetch so source authenticity is guaranteed by the hash, not the server.

5 Storage Pact Mechanism

5.1 Pact Formation

A storage pact is a bilateral agreement between two nodes to store each other's events. Formation follows a three-phase DVM-style (Data Vending Machine) negotiation:

1. **Request** (kind 10055): Node p broadcasts a storage pact request with its volume estimate, minimum pact count, and TTL.
2. **Offer** (kind 10056): Qualifying WoT peers respond with offers.
3. **Accept** (kind 10053): Both parties exchange private pact events and begin mutual storage.

Qualification requires: WoT membership (follows or followed-by), minimum account age (default 7 days to prevent Sybil pact formation), and a **follow-age requirement** of mutual follow history (relaxed to 72 hours during the first growth year, tightening as the network matures). This prevents gaming WoT edges instrumentally—an attacker cannot create a fresh identity, follow a target, and immediately form a reciprocal pact. There is no volume-balance requirement: each partner simply stores what the other produces up to a per-partner cap (Section 5, Capped Asymmetric Storage). Bootstrap pacts (one-sided, temporary) are exempt to avoid blocking new user onboarding.

5.2 Capped Asymmetric Storage

Pacts do not require matched data volumes. Instead, each partner stores whatever the other produces, bounded by a per-partner cap:

$$\text{store}(p \text{ for } q) = \min(\text{volume}(q),\ \text{CAP}) \tag{5}$$

where CAP defaults to ∼10 MB/month/partner. This bounds each partner's worst-case storage cost without policing the balance between the two, so a prolific poster and a quiet lurker can hold a pact without renegotiation. Removing volume matching (and its drift-triggered renegotiation) eliminates a churn source and readmits the lurker majority to the storage economy.

5.3 Pact Topology

Each node maintains a single set of active pact partners—partners that are regularly challenged and expected to serve data on request. When a partner sustains a Failed reliability band it is replaced by forming a fresh pact (renewal-by-default); there is no separate standby reserve.

Target-Based Formation

The protocol forms pacts toward a simple target rather than an emergent equilibrium: **target 20 active pacts, floor 12**. A node keeps forming pacts (via the request/offer/accept negotiation

of Section 5) until it reaches the target, and does not drop below the floor. Formation is driven by simple replacement plus a diversity heuristic; the pact-formation lifecycle dissolves into the three-phase adoption arc (Section 7), so the only two lifecycles in the protocol are that arc and the four-state pact FSM (Forming → Active → Failing → Ended). Earlier drafts prescribed an emergent count via a Poisson-binomial "comfort condition"; that machinery has been removed (ADR 016) in favor of the fixed target, which is simpler to reason about and to validate.

Keeper excess capacity and PACT_FLOOR: High-uptime Keepers need fewer partners for their own availability than intermittent Witnesses do. Left to pure self-interest, Keepers would stop accepting pacts early, starving Witnesses of the always-on partners they most need. The floor of 12 is the structural fix: every node maintains at least 12 active pacts regardless of personal comfort, so Keepers carry excess capacity that provides the availability Witnesses depend on. This is the storage-layer expression of the pay-it-forward moral engine.

Functional diversity constraint: Clients enforce a minimum Keeper (full node, 90%+ uptime) ratio among active pact partners: at least 15% of active pacts must be Keepers (≈ 3 of 20). A pact set with zero Keepers has no always-on storage—all data becomes unavailable during overnight hours when Witnesses sleep. When the Keeper ratio drops below the minimum, the client prioritizes recruiting Keeper partners.

Uptime complementarity: Peer selection minimizes uptime overlap across the pact set. Clients compute a *pact set coverage score*: for each hour of the day, count how many pact partners are typically online (derived from challenge-response timestamps over a rolling window). The coverage score is the minimum hourly count across all 24 hours. Target: ≥ 3 (at least 3 partners online during every hour). When evaluating new pact offers, prefer partners whose uptime pattern fills existing coverage gaps. This catches a failure mode that geographic diversity alone misses: peers in adjacent timezones with 80%+ uptime overlap due to similar schedules.

Correlated failures: The independent-failure assumption is optimistic. Shared timezones, ISPs, and OS updates introduce correlation. Under a beta-binomial model, correlation $\rho = 0.10$ requires $\sim$2.6$\times$ more pacts; $\rho = 0.20$ requires $\sim$10$\times$ more. Geographic diversity and uptime complementarity reduce ρ toward zero, keeping the required pact count near the target.

5.4 Data Availability Verification

Pact partners are verified through periodic challenge-response exchanges (kind 10054):

Hash challenge: The challenger specifies a range of event sequence numbers and a nonce. The partner computes $H(\text{events}[i..j] \| \text{nonce})$ and returns the hash. This proves possession without transferring full events. The exact serialization is: SHA-256(canonical_json(e_i)$\|$canonical_json(e_{i+1})$\| \ldots \|$canc where `canonical_json` is Nostr's event serialization format (the `[0, pubkey, created_at, kind, tags, cor` array used to compute event IDs). The nonce is 32 random bytes. Events are ordered by `(device_pubkey, seq)`—the same ordering used for Merkle tree construction.

Serve challenge: The challenger requests a specific event by sequence number and measures response latency. Consistently slow responses (>500ms) suggest the partner is proxying rather than storing locally. Flagged peers receive 3$\times$ challenge frequency.

This verifies data accessibility on demand, not local storage: a well-provisioned proxy that can re-fetch and serve the events quickly could pass both challenge types. The mechanism confirms that a partner *can produce* the data when asked, which is what retrieval actually depends on—not that it holds a dedicated local replica.

5.5 Reliability Scoring

Each node maintains per-peer reliability scores using an exponential moving average:

$$\text{score}' = \text{score} \cdot \alpha + \text{result} \cdot (1 - \alpha) \tag{6}$$

where $\alpha = 0.95$ and result $\in \{0, 1\}$. At roughly one challenge per day this gives a ∼3-week (≈20-sample) effective window, with recent challenges weighted more heavily. Scoring is presence-aware: only challenges issued while the partner was observed online count, so an intermittently-online *Witness* is not penalized for sleeping.

Score	Status	Action
$\geq 90\%$	Healthy	No action
70–90%	Degraded	Increase challenge frequency; begin seeking a replacement
50–70%	Unreliable	Actively replacing
$< 50\%$	Failed	Replace; drop only after Failed is sustained (∼14 days)

Age-biased challenge distribution: 50% of challenges target events from the oldest third of the stored range, 30% from the middle third, 20% from the newest third. This catches selective deletion of old data—a strategy where a peer keeps recent events (frequently requested) while discarding older events to save storage. Without age-biased distribution, a peer could maintain a >90% reliability score by storing only recent data.

Channel-aware challenge retry: When a challenge times out, the client distinguishes between "peer failed" and "communication channel failed." Before marking a challenge as failed, the client retries through an alternative relay (selected from NIP-65 relay list or known mutual relays). If the retry succeeds, the original relay is flagged as unreliable; future challenges for this peer use the working relay. This prevents a malicious or unreliable relay from causing pact churn by dropping NIP-46 messages.

5.6 Network Partition Handling

Network partitions (country-level shutdowns, WoT community splits, relay outages) are handled explicitly:

Detection: A partition is suspected when ≥3 pact partners in the same WoT cluster fail challenges within a 1-hour window while the client's own relay connectivity remains functional.

During partition: Reliability scoring is suspended for partition-affected peers. Pact replacement is not initiated. The client continues operating with its remaining reachable partners and relay fallback, which provide availability during the partition.

After partition heals: Challenge-response resumes with previously partitioned peers. Events created during the partition are reconciled via checkpoint comparison. Reliability scores are restored to pre-partition levels after 3 consecutive successful challenges. Peers that genuinely failed (not just unreachable) are replaced through standard mechanisms.

5.7 Pact Set Health

In addition to per-peer reliability, clients track aggregate pact set health:

- **Coverage score**—minimum number of online partners across all 24 hours (from uptime histograms). Target ≥ 3.
- **Overlap coefficient**—average pairwise uptime overlap. Lower is better (more complementary coverage).

- **Functional balance**—ratio of Keepers to total active pacts. Target $\geq 15\%$.

When the coverage score drops below 3 or the Keeper ratio drops below 15%, the client prioritizes pact replacement—selecting partners whose uptime profile fills identified coverage gaps.

5.8 Storage Obligations

The total storage obligation per user depends on the pact partner composition:

Formula F-03 (storage per user):

$$S = P \cdot (f \cdot \mathbb{E}[\text{size}] \cdot R \cdot D_{\text{full}} + (1 - f) \cdot \mathbb{E}[\text{size}] \cdot R \cdot D_{\text{light}}) \tag{7}$$

where P = number of active pacts (target 20, floor 12), f = fraction of Full node partners ($\sim$0.25), $\mathbb{E}[\text{size}] = 925$ bytes, R = events per day (default 25), D_{full} = complete history duration, $D_{\text{light}} = \min(\text{duration}, W)$.

5.9 Guardian Pacts

Guardian pacts extend the pact mechanism to support newcomers who have no Web of Trust presence. An established user (a *Guardian*) volunteers to store data for one newcomer (a *Seedling*) without any WoT membership requirement, and the pact is one-sided rather than reciprocal.

Guardian pacts use the same kind 10053 event with a `type: guardian` tag. Formation flow:

1. **Opt-in**: A Sovereign-phase node advertises guardian availability via kind 10055 with `type: guardian`.
2. **Match**: A Seedling with fewer than 5 pacts is matched (client-side or relay-assisted).
3. **Accept**: Both parties exchange kind 10053 with `type: guardian`.
4. **Store**: The Guardian stores the Seedling's events. Challenge-response verification (kind 10054) applies.

Expiry conditions: 90 days from formation, or the Seedling reaches Hybrid phase (5+ reciprocal pacts). Each Guardian holds at most one active guardian pact, bounding the storage cost to a single user's data volume.

6 Retrieval Protocol

6.1 Delivery Tiers

When a node needs to retrieve events from a followed author, the protocol attempts delivery through a three-tier cascade of increasingly expensive paths:

Tier 1—Local (instant): The node already stores the author's events locally, either through an active pact or from a previous read cache. Cost: zero network traffic. Latency: zero. This tier is expected to serve the large majority of reads for authors within a mature pact set.

Tier 2—Partner query (kind 10057): The node sends a NIP-44-encrypted direct data request to one of the author's known pact partners, addressed over iroh (Section 8). The request is signed and routed by pubkey; the partner responds with a data offer (kind 10058). Latency: $\sim$80ms + jitter.

Tier 3—Relay fallback: After a configurable timeout (default 30s), the node falls back to a traditional relay query. Latency: $\sim$200ms base + 50ms jitter. Relay use is reduced by the first

two tiers but not eliminated; it remains a permanent component of the delivery architecture, not merely a path of last resort.

Each successive tier is attempted only when the previous tier fails or times out, creating a natural cost gradient that keeps most traffic within the social graph.

Optional extension—WoT gossip: As a censorship-resistance extension (deferred to a future protocol version), a node may instead broadcast the Tier 2 request across its 2-hop WoT with TTL=3, letting any peer that holds the data respond. This trades the direct partner query's efficiency for resistance to targeted blocking of specific partners. It is not a required tier. A **BLE proximity tier** (offline, device-to-device) is likewise deferred; see Section 8.

6.2 Read Cache and Cascading Replication

A critical property emerges from the retrieval protocol: **reads create replicas**. When Bob fetches Alice's events from a storage peer, Bob now holds a local copy. When Carol subsequently requests Alice's events via gossip, Bob can respond—without being one of Alice's formal pact partners.

This creates cascading read-caches that replicate popular content across the follower base:

1. Alice's pact partners hold her events.
2. Each of Alice's followers who reads her events becomes an informal cache.
3. Subsequent readers find the data closer in the social graph.
4. Load scales with $O(\text{followers})$, not $O(\text{pact\_partners})$.

The read cache is bounded (default 100MB) and LRU-evicted.

Media retrieval follows the same cascade but at a per-file level. Clients first check local cache, then try the URL hint in the `media` tag, then query media pact partners (if any), then fall back to IPFS or relay-hosted caches. Hash verification ensures integrity regardless of source.

Content discovery beyond the WoT is intentionally relay-dependent. Content from authors outside the 2-hop WoT boundary is discoverable through relay queries (Tier 3). Clients subscribe to hashtag-filtered feeds from relays, which serve as curated discovery endpoints. The WoT boundary provides spam resistance and storage efficiency for the pact layer; relays serve the orthogonal function of broad content discovery.

This means content discovery is *permanently* relay-dependent—not a transitional artifact of the bootstrap phase. The protocol provides no decentralized mechanism for discovering content or authors outside the user's WoT. Relay dependency for content retrieval (fetching known authors' events) decays as pact coverage grows, but discovery remains a relay function indefinitely.

6.3 Request Privacy

Data requests (kind 10057) are signed Nostr events, NIP-44-encrypted to a chosen storage peer and addressed over iroh. An earlier draft of this protocol used a daily-rotating "request token" intended to hide the lookup target; that mechanism has been **removed**. It never hid the requester—a signed request necessarily exposes the asking pubkey to every forwarder and to the storage peer (per-source rate limiting and response routing both require it)—and the token was trivially reversible by anyone who already knew the target's pubkey.

The honest privacy story is therefore about *where* metadata flows, not about requester anonymity:

- The requester is always identified to the storage peer it queries. This is the same exposure as a Nostr relay query—but the metadata goes to a WoT peer of the requester's choosing, not to an arbitrary relay operator who logs every subscription.
- Because the request is NIP-44-encrypted to that peer, third parties in transit learn neither the target nor the content.
- Reader privacy relative to vanilla Nostr comes entirely from this narrower, self-selected metadata surface—WoT peers instead of arbitrary relays—not from any blinding of the requester.

6.4 Gossip Reach Analysis

With a mean degree of 20 peers and TTL=3, the gossip reach at each hop (accounting for 25% clustering coefficient) is:

```
hop 1: 20 nodes
hop 2: 20 * 20 * (1 - 0.25) = 300 nodes
hop 3: 300 * 20 * (1 - 0.25) = 4,500 nodes
```

Cumulative reach: ∼4,820 nodes within 3 hops.

This is a mean-field approximation that assumes uniform random neighborhood overlap. In highly clustered networks ($C > 0.5$), actual unique reach is substantially lower because 2-hop neighborhoods overlap heavily; at realistic online rates the unique *online* reach is on the order of a few hundred nodes. The protocol's primary delivery mechanism is pact-local instant reads followed by direct partner queries, not gossip propagation, with relay fallback (Tier 3) handling content beyond the pact set. Gossip is a secondary, optional mechanism (Section 6), not a required tier; its realized contribution to retrieval will be measured against the rebuilt simulator.

6.5 Rate Limiting and Gossip Hardening

The gossip layer implements per-source rate limiting to prevent amplification attacks. All hardening rules are enforced client-side—no relay modifications required:

- Kind 10055 (pact requests): 10 req/s per source pubkey.
- Kind 10057 (data requests): 50 req/s per source pubkey.

Rate limiting uses a sliding window counter per source. Excess requests are silently dropped. Combined with WoT-only forwarding, this bounds the gossip blast radius to the trust network while achieving high-probability delivery within it. Gossip provides probabilistic, not guaranteed, delivery. The relay fallback (Tier 3) is an integral component of the delivery mechanism, expected to carry a larger share of reads for dense or small-world topologies than for sparse hub-structured graphs, and a diminishing share as pact coverage matures. Relay fallback is not a path of last resort but a permanent component of the delivery architecture.

Request deduplication uses an LRU cache of 10,000 request IDs. Duplicate requests are dropped silently, preventing gossip loops and reducing amplification.

Gossip-forwarded events are limited to 64 KB per event. Events exceeding this limit are not forwarded through the gossip layer and must be retrieved directly from the author's storage peers or relays. This prevents gossip amplification attacks where an attacker within the WoT publishes arbitrarily large events that consume bandwidth across 3 hops.

6.6 Data Availability

The probability that all pact partners are simultaneously offline determines data unavailability:

Formula F-14 (all-pacts-offline probability):

$$P(\text{unavailable}) = (1 - u_{\text{full}})^{P \cdot f} \cdot (1 - u_{\text{light}})^{P \cdot (1-f)} \tag{8}$$

With representative parameters ($P = 20$, $f = 0.25$, $u_{\text{full}} = 0.95$, $u_{\text{light}} = 0.30$):

$$P(\text{unavailable}) = (0.05)^5 \cdot (0.70)^{15} = 3.125 \times 10^{-7} \cdot 4.747 \times 10^{-3} = 1.48 \times 10^{-9} \tag{9}$$

Under the assumption of independent failures, this represents approximately one-in-a-billion chance of complete data unavailability at any instant. This is an analytical *upper bound* of the independent-failure model on a mature, stable pact set—not a steady-state prediction. Real deployments depart from it in two directions the static formula does not capture: timezone correlation (pact partners in the same timezone sleeping simultaneously) and community correlation (shared ISP, shared OS updates) reduce effective redundancy, while pact churn creates transient windows with fewer than the target number of healthy partners. The protocol mitigates the first through geographic diversity and uptime complementarity in partner selection, and the second through renewal-by-default formation and presence-aware scoring.

Empirical validation is in progress: the simulator is being rebuilt against the current protocol (capped pacts, presence-aware scoring, renewal-by-default, three-tier retrieval), and measured availability figures will follow that rebuild.

6.7 Relay-Mediated Encrypted Channels (NIP-46)

NIP-46 [23] established that Nostr relays can serve as encrypted bidirectional message buses using NIP-44 encryption. While originally designed for remote signing ("Nostr Connect"), the underlying pattern—encrypted, bidirectional, relay-mediated communication—is a general-purpose primitive. Any two peers can establish an encrypted channel through a shared relay without exchanging IP addresses or requiring simultaneous online presence. The relay provides message confidentiality (it cannot read encrypted content) and IP address hiding between peers. However, the relay learns the communication graph—it sees both the sender's pubkey and recipient's pubkey (via the `p` tag), along with timing patterns of all operations. This metadata exposure is an inherent property of relay-mediated communication.

Gozzip leverages this infrastructure for three purposes:

- **Client-to-client communication.** Peers establish encrypted channels through any shared relay. No IP addresses are exposed, no NAT traversal is required, and no simultaneous online presence is needed. The relay queues encrypted messages until the recipient comes online.
- **Pact data transport.** Pact negotiation (kind 10053), challenge-response verification, and event synchronization flow through encrypted relay channels. This is especially valuable for intermittent-to-intermittent pairs where direct connections are unreliable.
- **Remote signing and pact delegation.** A pact management daemon (running on a desktop or VPS) handles storage obligations and challenge responses while signing keys remain on the user's personal device. The daemon requests signatures via NIP-46 only when needed—pact formation, challenge signing—separating key custody from availability obligations.

This reduces several categories of complexity:

- **NAT traversal**—relays handle connection brokering.
- **Direct connection requirements**—when using relay-mediated channels, peers never need each other's IP address. Direct connections (used optionally for bulk pact sync) do expose IPs to each pact partner.
- **Online simultaneity**—relays queue messages for offline recipients.
- **Custom transport infrastructure**—reuses existing NIP-46 relay implementations.
- **Key exposure risk**—signing stays on the user's device.

Relay-mediated channels do not replace the retrieval cascade (Section 6) but provide a reliable encrypted communication substrate for all peer-to-peer protocol messages. They complement the iroh transport (Section 8) for internet-based connectivity, offering an additional path that works through the existing Nostr relay network when direct connections are unavailable.

7 Flow Control

7.1 The Capacity Problem

Van Renesse et al. [2] demonstrated that anti-entropy protocols have bounded capacity: under high update load, gossip messages cannot carry all required deltas, and update latency grows without bound. Our protocol faces an analogous constraint: each node's gossip bandwidth is finite, and the rate of storage pact requests, data requests, and event deliveries must be controlled.

7.2 Pact-Aware Priority

Rather than the Scuttlebutt reconciliation's approach of ordering deltas by version number, our protocol orders gossip forwarding by social proximity:

1. **Active pact partners**: forwarded immediately, no queuing.
2. **1-hop WoT**: forwarded with standard priority.
3. **2-hop WoT**: forwarded when capacity is available.
4. **Beyond WoT**: never forwarded.

This is analogous to scuttle-depth ordering [2]—the protocol prioritizes propagating information for the peers most relevant to the local node, rather than being "fair" across all participants.

7.3 Three-Phase Adoption

The protocol includes a built-in flow control mechanism through its adoption phases:

Phase	Pact Count	Behavior
Bootstrap	0–5	Publish to relays primarily; form pacts as available
Hybrid	5–15	Publish to both relays and peers; fetch from peers first
Sovereign	15+	Storage peers primary; relays serve as delivery infrastructure with reduced data

New users begin in Bootstrap phase with full relay dependence. As they form pacts, traffic gradually shifts from relays to peer mesh. This provides natural flow control: the rate at which a new node joins the gossip network is limited by the rate at which it forms pacts, preventing gossip overload from sudden influx.

7.4 Pact Renegotiation Jitter

When a user's activity changes and pact renegotiation is needed, the protocol introduces random jitter (0–48 hours) before broadcasting replacement requests. Because a node keeps a floor of active pacts and drops a partner only after sustained failure (Section 5), the remaining partners continue serving data during this delay. This prevents renegotiation storms—the gossip equivalent of TCP's synchronized loss recovery problem [5].

7.5 Deletion Requests and Content Reporting

Deletion requests (kind 10063) provide a protocol-level mechanism for authors to request removal of their own events—addressing GDPR Article 17 ("right to erasure") compliance. A deletion request is signed by the root key or governance key (not device keys, preventing a compromised device from wiping history) and references specific event IDs. Pact partners, read-caches, and relays honor deletion requests on a best-effort basis. Deletion in a decentralized system is inherently best-effort—signed events may have been copied beyond the author's reach—but pact partners that systematically ignore deletion requests can be flagged and replaced through standard pact renegotiation. Compatible with NIP-09 (kind 5).

Content reports (kind 10064) enable users to report content to relay operators and community moderators. Reports carry a category tag (`spam`, `harassment`, `illegal`, `csam`, `impersonation`) and are published to the reporter's relays and the reported author's relays via NIP-65. Relay operators act at their discretion. Pact partners receiving credible `csam` or `illegal` reports may delete the content and drop the pact with no reliability score penalty. Compatible with NIP-32 labeling services for third-party content filtering.

7.6 Push Notifications

Mobile devices cannot maintain persistent connections. Push notifications (kind 10062) register a user's encrypted push token with a *notification relay*—a lightweight service that monitors for events matching the user's filters and sends a "sync now" wake-up signal via APNs or FCM.

The push token and filter preferences are NIP-44 encrypted to the notification relay's pubkey—relays storing this event see an opaque blob. The push payload contains no message content; the app wakes, syncs from relays and pact partners, and generates a local notification with actual content. Users register with 2–3 notification relays for redundancy. Token rotation publishes an updated kind 10062; deregistration publishes one with empty content.

8 iroh Transport: Direct Node-to-Node Connectivity

8.1 Motivation

The protocol as described needs a way for two nodes to exchange pact data, challenges, and retrieval requests directly, without routing every byte through a relay. Relay-mediated channels (Section 6) cover the case where peers cannot reach each other, but a maintained peer-to-peer transport is what lets the retrieval cascade keep traffic inside the social graph. The protocol builds on **iroh** [6], a production peer-to-peer library that provides authenticated, encrypted, direct connections between nodes.

8.2 iroh Architecture

iroh provides three capabilities the protocol relies on:

QUIC transport: All node-to-node traffic runs over QUIC (encrypted, multiplexed, connection-migrating). A connection survives the device changing networks (WiFi to cellular), because it is bound to the peer's identity rather than its IP address.

Ed25519 endpoint identity: Each node has a stable iroh endpoint keypair (Ed25519). The endpoint public key *is* the network address a peer dials—no separate address allocation, no DNS. Because a Gozzip user's protocol identity is a secp256k1 key while the iroh endpoint is Ed25519, the two are bound by a signed **kind-10070 cross-key attestation** (Section 4), so a pact partner can verify that a given iroh endpoint genuinely belongs to the expected root identity.

Relay-assisted hole punching: When two nodes are behind NATs, iroh uses lightweight relays only to coordinate a direct connection (hole punching), then upgrades to a direct path where possible and falls back to relayed QUIC where not. This is transport-level relaying for connectivity, distinct from Nostr relays' role in storage and discovery.

8.3 Identity Binding

The critical integration point is that a node's protocol identity and its transport identity are cryptographically linked, not merged. The secp256k1 root pubkey names the user; the Ed25519 iroh endpoint names the connection; the kind-10070 attestation (secp256k1 $\leftrightarrow$ Ed25519) ties them together. A storage pact partner is therefore simultaneously a directly-dialable transport peer, with no trusted address-mapping oracle required.

8.4 Gossip and Queries over iroh

The protocol's peer messages—kind 10057 partner queries, kind 10055 pact requests, challenge-response exchanges, and the optional WoT gossip extension—are carried over iroh connections:

Scenario	iroh path	Behavior
Both nodes reachable	Direct QUIC (hole-punched)	Full-speed direct queries and sync
One/both behind strict NAT	Relay-assisted QUIC	Same messages, relayed connectivity
Device changes network	Connection migration	Session survives WiFi$\leftrightarrow$cellular handoff

iroh's own membership and broadcast layer (a HyParView-style partial-view membership protocol with PlumTree-style epidemic broadcast) provides the substrate for the optional gossip extension, so the protocol does not reimplement peer sampling or message dissemination.

8.5 Offline-First Operation (Future Multipath)

Transports beyond the internet are a *future* direction layered on the same identity model, not part of protocol v1. They reuse the Ed25519 endpoint identity and kind-10070 binding unchanged, so adopting them later requires no change to the protocol's identity or retrieval layers:

1. **BLE proximity (deferred)**: device-to-device exchange for offline, disaster, and activism use. No iroh BLE transport exists today, so this is deferred (see ADR 011). A future BLE layer could interoperate with BitChat [24], an open-source protocol for encrypted peer-to-peer messaging over BLE, adopting its well-engineered primitives—Noise Protocol session management, compact binary framing for low-bandwidth links, Bloom filter deduplication, and fixed-size packet padding—rather than reimplementing them.
2. **Store-and-forward queuing**: when no transport is available, events are queued locally (up to 1,000 events or 50MB) and auto-drain when any transport becomes available.

3. **Tor and radio multipath (future)**: extending iroh endpoints to be reachable over onion circuits or low-bandwidth radio links, with geohash-tagged ephemeral subkeys (not linked to identity) enabling nearby-device discovery without revealing identity.

The relationship with BitChat is complementary. BitChat excels as a real-time ephemeral mesh—designed for scenarios like protests or disaster zones where there is no internet and no persistent identity is needed. Our protocol adds the persistence layer: storage pacts that keep data alive across offline periods, a Web of Trust that governs propagation, and a social graph that serves as long-term infrastructure. If the deferred BLE work lands, BitChat handles immediate local communication while our protocol handles sovereign social networking—together a complete stack from Bluetooth radio to persistent social media.

8.6 Interplanetary Compatibility

The architectural choices described above—self-authenticating events, store-and-forward queuing, checkpoint reconciliation, explicit partition handling—happen to satisfy the requirements of delay-tolerant networking (DTN). A DTN transport adapter for Bundle Protocol (RFC 9171), layered under iroh's identity-addressed endpoints, would allow the same events, pacts, and gossip mechanisms to operate over interplanetary links.

Cross-planetary pacts should *not* form: solar conjunction (2-week blackout every 26 months) and 3–22 minute one-way latency make challenge-response impractical and pact availability negligible. Instead, each planetary community forms its own pact mesh, and cross-planetary data exchange uses data mules (ships, scheduled transmissions) whose crew act as cascading read-caches—exactly the mechanism described in Section 6. The WoT 2-hop boundary naturally contains local gossip within each planet.

The primary attack vector in an interplanetary topology is *WoT bridge compromise*—the 50–100 users who follow people on both planets form chokepoints for cross-planetary information flow. Bridge diversity monitoring (tracking how many independent bridges deliver content from a given author) and cross-bridge checkpoint verification provide detection. This is a structural vulnerability of any socially-routed interplanetary network.

This is not an interplanetary protocol—it does not include planetary routing, scheduled contacts, or DTN-specific mechanisms. It is a protocol whose terrestrial architecture is *compatible* with interplanetary requirements, and could be extended for that use case via a DTN transport adapter and latency-adapted timeouts.

8.7 The "Pub Server" Problem, Solved Differently

Scuttlebutt [7] identified the fundamental tension in peer-to-peer social networks: mobile devices go offline, but content must remain available. Scuttlebutt's solution was "pub servers"—always-on nodes that replicate data. But pub servers are relays by another name.

Our protocol addresses this through two mechanisms that a maintained direct transport makes feasible:

1. **Full nodes as pact partners**: The target 25% of network participants that are always-on serve as full-history storage peers (the protocol is *intended* to degrade to as low as ∼5%—analytically plausible but not simulated). Unlike pub servers, they have bilateral obligations enforced by challenge-response.

2. **Identity-addressed reachability**: When a mobile device changes networks or sits behind a NAT, its iroh endpoint remains dialable via connection migration and relay-assisted

hole punching. A partner does not need to rediscover an address; it dials the same endpoint identity.

9 Related Work

9.1 Anti-Entropy and Epidemic Protocols

The foundational work on epidemic algorithms for replicated database maintenance [8] established that updates spread in $O(\log N)$ rounds through random peer selection. Van Renesse et al. [2] extended this with the Scuttlebutt reconciliation mechanism and AIMD-based flow control. Our protocol adopts the epidemic spreading model but replaces random peer selection with WoT-weighted selection, sacrificing theoretical convergence speed for trust-bounded propagation.

9.2 Scuttlebutt / Secure Scuttlebutt

Secure Scuttlebutt (SSB) [7] proved the viability of gossip-based social networking with local-first storage and append-only feeds. Our protocol builds on SSB's core insight but differs in three ways:

1. **Nostr key model**: SSB uses ed25519 feeds tied to devices; we use secp256k1 keypairs with a delegation hierarchy that supports multi-device and key rotation.
2. **Bilateral pacts vs. unilateral replication**: SSB's pubs replicate unilaterally; our pacts enforce reciprocal obligation with data availability verification.
3. **WoT-bounded gossip**: SSB's gossip has no explicit trust boundary; our protocol restricts forwarding to 2-hop WoT distance.

9.3 Data Availability Verification

The challenge-response mechanism draws from proof-of-storage work: Filecoin's Proof of Replication and Proof of Spacetime [9], and Arweave's Succinct Proofs of Random Access [10]. Our mechanism verifies that the pact partner can *produce* the data on demand at challenge time. Unlike Filecoin's PoRep, it does not prove continuous or dedicated local storage—a malicious partner (or a well-provisioned proxy) could theoretically re-fetch data before responding. The 500ms latency check on serve challenges is a weak heuristic against this strategy. This is a deliberate trade-off: social trust (WoT peers with relationship incentive) reduces the need for cryptoeconomic enforcement, at the cost of being less robust against adversarial pact partners than Filecoin's scheme. Accordingly we describe this as data *availability* verification rather than proof of storage.

9.4 iroh and Peer-to-Peer Transport

iroh [6] provides the maintained peer-to-peer transport layer our protocol builds on: QUIC connections addressed by Ed25519 endpoint identity, relay-assisted hole punching, and connection migration across networks. Its HyParView/PlumTree membership-and-broadcast layer is complementary to our WoT-based routing: iroh handles physical reachability and epidemic dissemination, while the WoT layer decides social relevance and which peers to query.

9.5 BitChat and BLE Mesh Messaging

BitChat [24] implements encrypted peer-to-peer messaging over Bluetooth Low Energy using a four-layer protocol stack: application, session, Noise Protocol encryption, and trans-

port. Its design goals—confidentiality, authentication, forward secrecy, and resilience in lossy environments—align with the requirements of a future BLE proximity layer for our protocol (deferred; see ADR 011). BitChat's Noise XX handshake provides mutual authentication and forward secrecy without requiring prior key exchange, its compact binary framing minimizes bandwidth on constrained BLE links, and its Bloom filter gossip deduplication prevents routing loops efficiently. Should that layer land, our protocol would adopt BitChat's primitives rather than reimplementing them, and extend the stack with persistent storage (pacts), trust-weighted routing (WoT gossip), and long-term identity (key hierarchy with N-of-M recovery contacts)—capabilities that BitChat intentionally omits in favor of ephemeral, infrastructure-free messaging.

9.6 Local-First Software

The Ink & Switch research group's work on local-first software [11] articulated the principles our protocol implements: data ownership, offline operation, and collaboration without mandatory servers. Our contribution is extending these principles to a social networking context where the "collaborators" are an entire social graph, not a small team.

9.7 Infrastructure Benchmark: Nostr and Mastodon

The two most prominent decentralized social protocols—Nostr [1] and Mastodon/ActivityPub [12]—represent fundamentally different architectural choices. Nostr separates identity from infrastructure but depends on relay servers. Mastodon binds identity to infrastructure through domain-coupled accounts. Our protocol makes the social graph itself the infrastructure, while maintaining interoperability with both: Nostr events work natively (same key model, same relays), and public content can be exported to ActivityPub format for Mastodon federation via bridge services (protocol-specific features such as pacts and WoT routing are not bridgeable). The following tables provide a structured comparison.

Identity and Portability

	Nostr	**Mastodon**	**Gozzip**
Identity	secp256k1 keypair	`user@instance` (domain-bound)	secp256k1 + key hierarchy
Portability	Full—identity is the keypair	Partial—followers migrate; posts do not	Full—same Nostr key model; public content exportable to ActivityPub/AT Protocol/RSS via bridges
Multi-device	Share private key or NIP-26	Native—server holds session	Root key delegates to device subkeys
Recovery	None—lose key, lose identity	Admin can reset password	Root key revokes subkeys; root loss recoverable via an N-of-M recovery-contact scheme (kind 10060/10061) with 7-day timelock

	Nostr	Mastodon	Gozzip
Where data lives	Relay servers	Home instance (PostgreSQL)	User's device + ∼20 pact partners (target 20, floor 12)
Who controls	Relay operators	Instance admin	User + bilateral pact partners
Redundancy	Manual multi-relay publishing	None—single home instance	Target 20 active pacts (floor 12); analytical upper bound $\sim 1.48 \times 10^{-9}$ unavailability for a stable pact set (empirical validation pending simulator rebuild)
If infra dies	Events lost unless duplicated	All posts from instance lost	Pact partners retain copies

Data Storage and Ownership

Content Distribution

	Nostr	Mastodon	Gozzip
Mechanism	Client pulls from relays via WebSocket (REQ/EVENT)	Server pushes to followers' instances via HTTP POST to shared inbox	Three-tier cascade: local pact (Tier 1) → direct partner query (Tier 2) → relay fallback (Tier 3); WoT gossip is an optional extension
Discovery	Client connects to author's outbox relays (NIP-65)	Federated timeline, relay servers, hashtag trends	Direct partner queries within the WoT; relays for discovery beyond it; 2-hop boundary for the optional gossip extension
Read privacy	Relay sees all subscriptions and read patterns	Instance admin can see all activity; DMs stored unencrypted	Requests are NIP-44-encrypted to a chosen WoT peer; the requester is identified to that peer (as in a Nostr relay query) but the metadata reaches self-selected WoT peers, not arbitrary relay operators
Offline	Not supported	Not supported	Direct connectivity over iroh; BLE proximity and radio/Tor multipath are future extensions (deferred)

Censorship Resistance

	Nostr	Mastodon	Gozzip
Censorship surface	Individual relays can drop events; mitigated by multi-relay publishing	Instance admin has unilateral suspend/delete power; domain blocks sever federation	No single point—data distributed across ∼20 WoT peers (target 20, floor 12)
Content filtering	Per-relay policies; NIP-42 auth gating; proof-of-work spam deterrence (NIP-13)	Per-instance admin moderation; community-level domain blocklists	WoT filtering—gossip propagates only within 2-hop trust boundary; no global moderation authority
Account risk	Low (self-sovereign keypair) but relays can refuse to serve events	High—admin can suspend with no technical appeal mechanism	Very low—pact partners hold data under bilateral obligation

Scalability and Costs

	Nostr	Mastodon	Gozzip
Cost bearer	Relay operators (shifted to users via paid relays, ∼$5–20/mo)	Instance operators (media cache: 10–20 GB/day with relay subscriptions)	Distributed across participants—capped asymmetric pacts (∼10 MB/month/partner)
Bandwidth	Variable; duplicate events across relays multiply client bandwidth	Federation delivery: one HTTP POST per remote instance with followers	Scales with pact count and event rate; bounded by the per-partner storage cap
Popular content	Each relay serves independently; no coordinated caching	Boosts replicate to each instance separately	Cascading read-caches: reads create replicas, scaling O(followers)
Infrastructure	∼1,000 relays (471 public, 191 restricted); concentration on high-traffic relays	∼26,000 instances; mastodon.social dominates with 348k+ accounts	Target 25% Full nodes (always-on), 75% Light nodes (intermittent); intended to degrade to ∼5% full nodes (not simulated); no central infrastructure

Architectural Summary

Nostr's design cleanly separates identity from infrastructure, but relays remain the storage and distribution layer—recreating a dependency surface analogous to the platforms it replaces. The outbox model (NIP-65) and negentropy sync (NIP-77) improve relay coordination but do not eliminate relay dependence.

Mastodon bundles identity with infrastructure. The `user@instance` model means your account's survival depends on your instance operator's continued goodwill, funding, and uptime. FEP-ef61 (portable objects) and FEP-8b32 (object integrity proofs) aim to decouple identity from instances, but these remain proposals.

Our protocol addresses both failure modes by making the social graph the infrastructure layer. Storage pacts distribute data custody across WoT peers with cryptographic verification. The relay's role shifts from gatekeeper to delivery infrastructure with reduced custody—reduced, not

optional. Critically, the protocol works with standard Nostr relays without any modifications—all Gozzip event kinds are valid Nostr events, and all protocol intelligence (gossip forwarding, encrypted request routing, WoT filtering, device resolution) lives in clients. Because events are self-authenticating (signed JSON), public content (posts, reactions, reposts) can be exported to ActivityPub, AT Protocol, and RSS/Atom via bridge services. Protocol-specific features (pacts, WoT routing, device delegation, encrypted DMs) are not bridgeable. Cross-protocol identity linking uses signed attestations.

The trade-off is protocol complexity: pact negotiation, challenge-response verification, WoT computation, and multi-tier retrieval are substantially more complex than Nostr's REQ/EVENT or Mastodon's HTTP POST delivery.

The honest gap: our protocol is grounded in analytical modeling, and its empirical validation is in progress—the discrete-event simulator is being rebuilt against the current protocol (capped pacts, presence-aware scoring, renewal-by-default, three-tier retrieval), and measured figures will follow that rebuild. Nostr has ∼1,000 relays and real users; Mastodon has ∼26,000 instances and ∼1.2M monthly active users. The architectural claims require production-scale validation.

10 Conclusion

We have presented an open, censorship-resistant protocol for social media and messaging that returns data custody from server infrastructure to the social graph. The protocol inherits Nostr's proven primitives—secp256k1 identity, signed events, relay transport—and adds a storage and retrieval layer where users own their data. Because events are self-authenticating, public content (posts, reactions, reposts) can be exported to ActivityPub, AT Protocol, and RSS/Atom via bridge services; protocol-specific features (pacts, WoT routing, device delegation, encrypted DMs) are not bridgeable, and cross-protocol identity linking uses signed attestations. The protocol works with standard Nostr relays without any modifications—all protocol intelligence lives in clients.

The protocol's design mirrors human social dynamics: reciprocal storage pacts parallel reciprocal friendships, WoT-filtered gossip parallels trust-based information sharing, and the Full/Light node distinction mirrors the inner-circle/outer-circle structure of human communities.

The three-tier retrieval cascade (local pact storage, direct partner query, relay fallback, with WoT gossip as an optional extension) is designed so that the majority of reads are served from local pact storage, with relay fallback reserved for cold-start and beyond-graph content and its share expected to decay as pact coverage matures. The relay's role shifts from gatekeeper to delivery infrastructure with reduced custody—useful during bootstrap and for distant authors, and relay-as-Keeper is in fact the intended Genesis deployment model. This is reduced relay custody, not optional infrastructure.

The iroh transport gives each node direct, identity-addressed connectivity for pact data and retrieval, with mesh radio, Bluetooth, and Tor multipath as future extensions that reduce internet dependence. The shared identity model keeps this seamless: a user's secp256k1 root identity is bound to their Ed25519 iroh endpoint by a signed kind-10070 attestation.

The protocol addresses practical deployment concerns alongside the core architecture: a target-based formation model (target 20 active pacts, floor 12) forms pacts by simple replacement and a diversity heuristic, dissolving the pact-formation lifecycle into the three-phase adoption arc; content-addressed media references keep pact obligations tractable (a peer-to-peer media redundancy layer is deferred); deletion requests (kind 10063) provide GDPR-compatible erasure semantics; content reporting (kind 10064) enables community moderation without centralized authority; push notifications (kind 10062) bridge the gap between mobile device constraints

and real-time communication; and an N-of-M recovery-contact scheme (kinds 10060/10061) resolves the root key loss problem that plagues all keypair-based identity systems. The protocol's architectural foundations—self-authenticating events, store-and-forward, checkpoint reconciliation, partition handling—are structurally compatible with delay-tolerant networking, enabling potential extension to interplanetary use via a DTN transport adapter without core protocol changes.

The honest assessment: this architecture does not eliminate the need for always-on infrastructure. Full nodes—*Keepers*—(target 25% of the network; the protocol is *intended* to degrade to as low as ∼5%, a regime that is analytically plausible but not simulated) are the protocol's equivalent of reliable friends who are always available. The difference is structural: these nodes operate under bilateral obligations enforced by challenge-response, not under unilateral control of a platform operator. The infrastructure exists, but it is owned by the social graph.

The hard problems remain. Graph bootstrap for new users requires initial relay dependence and Genesis Guardian infrastructure (see design documentation). A sufficient full-node ratio (at least ∼5%) must emerge organically through incentives, not be mandated. DM keys rotate periodically (default every 90 days), which bounds the blast radius of a device- or epoch-key compromise—but this is not forward secrecy: because epoch keys are derived deterministically from the root, root-key compromise yields all past and future epoch keys. The N-of-M recovery-contact scheme (kinds 10060/10061) is a pre-launch requirement, not a future enhancement: without it, root key loss is permanent identity death. These are engineering challenges, not architectural ones—the social graph is a viable foundation for decentralized infrastructure.

11 Implementation Roadmap

This section addresses a different question: how does a real network get from today's relay-dependent world to the sovereign architecture described above? The answer is not revolution but transition—a sequence of phases where each delivers concrete value before the next begins, and where the user experience changes only when the underlying infrastructure has already proven itself.

11.1 Day-One Value

The pact layer requires network density to function, but the protocol provides immediate value before any pacts exist:

- **Multi-device identity**—root key + device delegation (kind 10050) eliminates Nostr's shared-key model. Device compromise is contained; revoke the device, keep the identity.
- **Encrypted DM threading with key separation**—DM encryption targets a dedicated, rotated key rather than the root key. Per-device DM capability flags limit the attack surface.
- **Social recovery**—N-of-M threshold recovery with a 7-day timelock works from the moment recovery contacts are designated.
- **Per-device hash chains**—every device-signed event carries `seq` and `prev_hash` tags, enabling integrity verification from the first event.
- **Fork-detecting profile/follow merge**—the `prev` tag on replaceable events enables automatic fork detection and deterministic merge across devices.

The first Gozzip client should be adopted because it is the best Nostr client available—multi-device, better DMs, social recovery, hash chain integrity. Data sovereignty is not a gate; it is a

gradient that increases as the network grows.

11.2 Design Philosophy: Transition, Not Revolution

The protocol must coexist with relays, not replace them on day one. Nostr's relay infrastructure works—and the protocol is designed so that standard Nostr relays require zero modifications. All Gozzip event kinds are valid Nostr events. Users have accounts, follow lists, and reading habits built on relays. Any viable deployment path must treat this as a starting point, not an obstacle.

The transition follows three principles:

1. **Invisible first, visible later.** The pact layer works silently underneath the existing client experience. Users should see zero behavioral change during Phase 1—no new UI, no configuration, no education required. Storage decentralization happens in the background before retrieval decentralization changes the read path.

2. **Each phase delivers value independently.** Phase 1 (storage redundancy) is valuable even if Phase 2 never ships. Phase 2 (fallback retrieval) is valuable even if Phase 3 never ships. No phase depends on the completion of a later phase for its value proposition to hold.

3. **Hardware trends are an ally.** What costs 10 MB/day and 5% battery today will cost nothing in three years. Mobile storage doubles every two years. 5G bandwidth is becoming ubiquitous. The protocol should be designed for the devices that will exist at deployment, not the devices that exist during simulation. Constraints that feel tight today will be invisible by the time Phase 3 deploys.

A critical framing: not everyone needs to be a full node. Full nodes don't need 100% uptime. The protocol's redundancy model means your data has *someone* holding it when you're offline—and with a target of ∼20 pact partners, the analytical model puts the chance that all of them are simultaneously unavailable very low (Section 6). The goal is not universal participation at maximum capacity; it is sufficient redundancy across realistic participation patterns.

11.3 Phase 1: Decentralize Storage

Goal: Every user's data exists in multiple places, not just relays.

The client works exactly like today—reads from relays, writes to relays. Nothing changes in the user-facing read or write path. In the background, the pact layer activates:

- **Pact formation begins** as users build their social graph. Mutual follows create WoT edges; the pact negotiation protocol (Section 5) finds pact partners within the WoT boundary.
- **Pact partners silently replicate** each other's events. When Alice publishes a note to her relay, her 20 pact partners also receive and store it. This happens through the existing gossip channel—no additional user action required.
- **Challenge-response runs quietly.** Data availability verification (Section 5) operates on a background schedule. Users never see it. Failed challenges trigger partner replacement through renewal-by-default formation.
- **No UX change. No user education needed. It just works.**

The value delivered is straightforward: if a relay goes down, deletes content, or gets censored, the data still exists on pact partners. This is *backup*, not *retrieval*—the relay remains primary

for reads. But the single point of failure is eliminated.

Mobile cost during Phase 1 is modest. A light node storing 5 partners' recent events (30-day window) requires approximately 150 KB/day of sync traffic—less than loading a single web page. Storage obligation is bounded by the checkpoint window and the per-partner storage cap (Section 5).

Key metric: Data survival rate when relays fail. With mature pacts, the vast majority of reads should succeed from local pact storage without any relay involvement.

11.4 Phase 2: Opportunistic Retrieval

Goal: When relays fail, try pact partners before giving up.

The relay remains the primary read path. The change is what happens on failure:

- **Relay failure triggers pact fallback.** On relay timeout, 404, or offline status, the client silently tries pact partners of the target author before displaying an error. This activates Tier 2 (direct partner query) of the three-tier cascade (Section 6) ahead of the Tier 3 relay path.
- **Users experience fewer failures.** Instead of "relay unreachable" errors, content loads anyway—from a pact partner who holds the author's data. The user sees "that post loaded" instead of an error message.
- **Clients learn which pact partners are reliable and fast.** Successful retrievals build a local routing table: for each author, which of their pact partners responded fastest? This makes subsequent direct partner queries (Tier 2) faster; it is a local hint cache, not a separate delivery tier.
- **This is the fallback phase—relay-first, pact-second.** The read path is: try relay → on failure, try known pact partners → on failure, display error.

Key metric: Failure rate reduction compared to relay-only operation.

11.5 Phase 3: WoT-Native Reads

Goal: Reads from close social contacts go to pact partners first, relays second.

This phase inverts the read priority for content within the user's Web of Trust:

- **Direct WoT reads (mutual follows) try pact partners before relays.** For authors who are likely pact partners—mutual follows with active storage agreements—the pact network is the primary read path. The relay becomes the fallback.
- **1-hop reads still try relays first.** Authors the user follows but who don't follow back are less likely to be pact partners, so the relay-first strategy remains appropriate for this tier.
- **The relay dependency decay curve kicks in.** Relay usage drops as pacts mature. This transition happens organically—no flag day, no coordinated switchover.
- **Trust-weighted gossip routing improves.** Forwarding decisions get smarter as the WoT graph stabilizes. Nodes learn which peers forward efficiently, which respond to gossip queries, and which are consistently offline. The gossip layer becomes a tuned routing mesh rather than a broadcast mechanism.

This is where the protocol's design goals—high instant delivery, near-total retrieval success—become the real-world experience. But the protocol doesn't need to start here. It earns this

performance by building through Phases 1 and 2, where the pact network proves itself as a reliable storage and fallback layer before being trusted as the primary read path.

11.6 Phase 4: Transport Independence

Goal: The protocol works without the internet.

- **BLE proximity (deferred)** would enable two phones in the same room to share events directly, device-to-device, without internet. No iroh BLE transport exists yet, so this is a future extension (see ADR 011), not part of protocol v1.
- **Store-and-forward queuing:** Events created offline are queued locally (up to 1,000 events or 50 MB) and drain automatically when any transport becomes available.
- **iroh multipath:** The protocol's peer messages already run over iroh (Section 8). Extending iroh endpoints to be reachable over Tor circuits or low-bandwidth radio links is a future direction that reuses the same Ed25519 endpoint identity—the message layer is transport-agnostic and would operate identically whether the underlying medium is fiber optic or a radio modem.
- **Geohash discovery with ephemeral keys:** Nearby users discover each other using geohash-tagged ephemeral subkeys that are not linked to their identity, enabling proximity-based exchange without surveillance.

This phase serves specific audiences: activism under censorship, disaster response when infrastructure fails, off-grid communities, privacy maximalists who want zero internet dependency. Not every user reaches this phase. That's fine. The protocol is useful at every phase—Phase 4 is an extension of capability, not a requirement for value.

11.7 Mobile as First-Class Citizen

The intuition that mobile devices can't participate in a storage protocol deserves scrutiny. Modern smartphones have capabilities that exceed what the protocol requires:

Resource	Available	Protocol Requires
Storage	128–256 GB	200–500 MB (power user, incl. indexes)
Daily bandwidth	1–5 GB (WiFi) / 2–5 GB (cellular)	4.1 MB/day (Light)
Background exec.	BGAppRefreshTask / WorkManager	Sync every 15–60 min
Battery	4,000–5,000 mAh	1–3% (bg sync only); 10–20% (bg + BLE); 10–15% (fg acti

The protocol's mobile architecture acknowledges that phones are intermittent participants, not always-on servers:

- **Light node by default.** The protocol is built for intermittent participation—challenge-response, pact obligations, and retrieval all account for variable availability.
- **Foreground mode:** Full gossip participation, real-time pact management, immediate challenge responses.
- **Background mode:** Periodic sync using OS-provided background task APIs (BGAppRefreshTask on iOS, WorkManager on Android). Challenge responses queue and resolve on the next background cycle. Heartbeats maintain pact partner awareness.
- **Sleeping mode:** Partners serve your data while you're offline. Challenges queue. You respond when you wake up. Presence-aware reliability scoring (Section 5) ensures that temporary absence doesn't trigger pact drops—a partner is judged only on challenges

issued while it was observed online, so offline periods consistent with the light node uptime assumption are tolerated.

Phone capabilities improve every year. What's a constraint today—background processing limits, storage tier pricing, cellular data caps—becomes trivial tomorrow. The protocol should be evaluated against the devices that will exist when Phase 3 deploys, not the devices available during Phase 1.

A "full node" doesn't need to be a phone. A desktop computer, a Raspberry Pi, or a $5/month VPS can serve as a full node—one technical friend running a full node serves 10–20 light nodes in their social circle as a high-uptime pact partner. This mirrors the social reality: every friend group has one person who runs the group server, hosts the shared drive, or keeps the archive. The protocol formalizes this role.

11.8 The Relay Doesn't Die

A critical framing error would be to read this roadmap as "relays go away." They don't. Relays remain valuable infrastructure with a changed role:

- **Discovery layer.** New users find content, authors, and communities through relays. The WoT graph doesn't help you find people you don't know yet—relays do.
- **Content curation.** Relay operators curate what they surface: filtering spam, promoting quality, organizing by topic. This is the relay's natural value proposition—editorial judgment, not data custody.
- **Performance accelerator.** For content from distant authors (beyond the 2-hop WoT boundary), relays provide faster access than multi-hop gossip. The relay is a CDN, not a database.
- **Bootstrap infrastructure.** New users start relay-dependent. That's by design, not a failure. The Bootstrap phase (Section 11) explicitly includes relay dependence, with pact formation gradually reducing it.

The protocol doesn't kill relays—it removes their monopoly on data custody. Relay operators become curators (choosing what to surface) rather than gatekeepers (deciding what exists). A relay that goes offline, changes its terms, or censors content no longer destroys the data it hosted—because that data exists on ∼20 pact partners (target 20, floor 12) who have bilateral obligations to preserve it.

Critically, the relays involved are standard Nostr relays. No custom software, no vendor lock-in. Every Gozzip event kind is a valid Nostr event that any relay will accept, store, and serve. Relay operators who wish to optimize for Gozzip traffic can implement optional accelerators—oracle resolution (caching device-to-root identity mappings), checkpoint delta delivery, gossip relay forwarding, NAT hole punching—but these are performance improvements, not requirements.

The transition is voluntary and invisible. Relay dependency drops naturally as pacts form. Users who never form pacts—because they're new, because they prefer relay-only operation, because their client doesn't implement the pact layer—continue to work exactly as they do today. The protocol adds capability without removing any existing functionality.

11.9 What We Don't Know Yet

This roadmap is a deployment plan for a protocol whose empirical validation is in progress: the simulator is being rebuilt against the current protocol (capped pacts, presence-aware scoring, renewal-by-default, three-tier retrieval), and measured figures will follow that rebuild. Two

modeled concerns the rebuilt simulator is meant to test are worth stating plainly: transient low-redundancy windows created by pact churn, and the risk that pact dissolution outpaces formation and contracts the pact set. Several further questions remain open:

- **Minimum viable network size for Phase 2.** At what user count does opportunistic pact retrieval deliver measurably better failure rates than relay-only? The pact benefit is expected to emerge early, within the first days of pact formation, and to strengthen as coverage matures—but the crossover point and its dependence on real-world network density are among the figures the rebuilt simulator will measure.
- **Whether a sufficient full node ratio emerges organically.** High-uptime Keepers need fewer partners for their own availability than intermittent Witnesses do, so left to pure self-interest Keepers would stop accepting pacts before Witnesses have enough always-on partners. The floor of 12 addresses this structurally—Keepers carry excess capacity—but whether the resulting social incentive is sufficient in practice is an empirical question. Comparable decentralized systems achieve 0.1–5% always-on node participation. The relay-as-Keeper deployment model (the intended Genesis model) may help bridge this gap.
- **The lurker problem.** In typical social networks, 60–80% of users are consumers who produce little or no content. Under the old volume-matching rule these lurkers had little to offer and were effectively excluded from the bilateral pact economy. Capped asymmetric pacts (ADR 013) are designed precisely to readmit them: a lurker can store a partner's events (up to the cap) while producing little in return, since pacts no longer require matched volumes. Whether this fully closes the gap in practice—or whether the effective pact economy still concentrates among active creators—remains an empirical question. Either way, lurkers retain at least the experience they have today on vanilla Nostr.
- **The cooperative reach gradient is unquantified.** The cooperative equilibrium assumes that more pacts yield measurably more reach—incentivizing cooperation over defection. However, the actual magnitude of this gradient has not been measured. Whether a cooperator with 20 pacts gets appreciably more reach than a defector with a handful determines whether the cooperative equilibrium is robust or fragile. This differential needs explicit measurement against the rebuilt simulator.
- **Optimal challenge frequency on mobile.** Data availability challenges (Section 5) cost battery and bandwidth. Too frequent and mobile users drain resources; too infrequent and defection goes undetected. The current design uses exponential moving average scoring with $\alpha = 0.95$, giving a $\sim$3-week ($\approx$20-sample) effective window, but the optimal challenge interval for mobile devices specifically has not been empirically determined. Age-biased challenge distribution and channel-aware retry reduce wasted challenge traffic, but the base frequency remains a tuning parameter.
- **How the partner-hint cache performs in production.** The local routing table (which pact partner answered fastest for a given author) requires successful prior retrievals to populate. In a real deployment with persistent storage across sessions, it should warm up naturally—but its actual contribution to read performance is unknown.
- **Better retrieval algorithms.** The current retrieval cascade is a simple priority waterfall (Section 6). More sophisticated approaches—predictive caching based on follow-graph activity patterns, pre-emptive fetching of likely-to-be-read content, ML-driven routing optimization—may exist by the time Phase 3 deploys. The architecture should accommodate algorithm improvements without protocol changes, and the three-tier structure is designed to be extensible.

- **Economic sustainability of full nodes.** Full nodes bear disproportionate storage and bandwidth costs. Whether the social incentive (reliable storage reciprocity) is sufficient, or whether economic incentives (micropayments, premium services) are needed, remains to be determined.
- **Cooperative equilibrium fragility and pact churn.** The cooperative equilibrium is fragile above high defection rates, and a related concern is whether pact dissolution can outpace formation and contract the pact set over time. The current design attacks this risk directly: renewal-by-default keeps healthy pacts alive, capped asymmetric pacts (ADR 013) remove the volume-matching drift that used to trigger renegotiation, and presence-aware scoring (ADR 014) stops penalizing intermittently-online partners for sleeping. Together these are intended to keep the pact set stable at its target. Whether they succeed—and whether residual contributors such as premature reliability-driven drops or the divergence between modeled and real-world activity patterns still cause contraction—is a modeled concern the rebuilt simulator is meant to test. If contraction persisted in production, pact counts would settle below the target of 20, with correspondingly higher relay dependence.
- **DM metadata exposure.** NIP-59 gift wraps expose both sender (device pubkey, resolvable to root identity via kind 10050) and recipient (`p` tag) to relay operators handling DM traffic. Content is encrypted, but the communication graph is fully visible. DM relay rotation and batched delivery can partially mitigate timing analysis but do not hide the sender-recipient pairs.
- **Relay collusion threat.** Colluding relay operators controlling 60–80% of relay traffic can reconstruct near-complete social graphs from merged metadata and identify probable pact pairings through NIP-46 timing analysis. Direct partner queries (Tier 2) run over iroh rather than through relays, so read patterns are exposed to the queried WoT peer rather than to relay operators (Section 6); but bootstrap, discovery, and mailbox traffic still flow through relays. The protocol's primary defense is relay diversity (users connecting through many relays), but whether real-world relay concentration permits this diversity is an empirical question.

These gaps are not architectural blockers. The protocol's core mechanisms—pact formation, challenge-response verification, WoT-filtered gossip, tiered retrieval—rest on established analytical foundations, and their empirical validation is in progress as the simulator is rebuilt against the current design. The remaining unknowns are deployment parameters that require production data to resolve—engineering challenges, not architectural ones.

Acknowledgments

This protocol builds on the work of several open-source projects and their contributors:

- **BitChat** [24] — The BLE mesh transport layer (Tier 0) adopts BitChat's Noise Protocol session management, compact binary framing, and Bloom filter gossip deduplication rather than reimplementing these primitives. BitChat's design for encrypted peer-to-peer messaging over constrained BLE links directly informed the protocol's offline-first capabilities.
- **iroh** [6] — The maintained peer-to-peer QUIC transport the protocol builds on: direct, identity-addressed connections with relay-assisted hole punching and connection migration, extensible to future mesh/radio/Tor multipath.
- **Nostr** [1] — The event model, relay architecture, and NIP ecosystem provide the com-

patibility layer that makes incremental adoption possible. The protocol's events are valid Nostr events, and existing relays require no modification.

- **Secure Scuttlebutt** [7] — The insight that gossip-based social networking with local-first storage is viable, and the identification of the "pub server" problem that this protocol addresses through bilateral storage pacts.

References

[1] Nostr Protocol. "Nostr Implementation Possibilities." https://github.com/nostr-protocol/nips

[2] R. van Renesse, D. Dumitriu, V. Gough, C. Thomas. "Efficient Reconciliation and Flow Control for Anti-Entropy Protocols." LADIS Workshop, 2008.

[3] R. Dunbar. "Neocortex size as a constraint on group size in primates." *Journal of Human Evolution*, 22(6):469–493, 1992.

[4] M. Granovetter. "The Strength of Weak Ties." *American Journal of Sociology*, 78(6):1360–1380, 1973.

[5] V. Jacobson. "Congestion Avoidance and Control." ACM SIGCOMM, 1988.

[6] number0 (n0). "iroh: peer-to-peer QUIC connections that just work." https://github.com/n0-computer/iroh

[7] D. Tarr, E. Lavoie, A. Chen, J. Robinson. "Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications." ACM PLDI, 2019.

[8] A. Demers, D. Greene, C. Hauser, et al. "Epidemic Algorithms for Replicated Database Maintenance." ACM PODC, 1987.

[9] Protocol Labs. "Filecoin: A Decentralized Storage Network." 2017.

[10] S. Williams, V. Diiorio, S. Barski. "Arweave: A Protocol for Economically Sustainable Information Permanence." 2019.

[11] M. Kleppmann, A. Wiggins, P. van Hardenberg, M. McGranaghan. "Local-First Software: You Own Your Data, in Spite of the Cloud." Onward!, 2019.

[12] C. Webber, J. Tallon. "ActivityPub." W3C Recommendation, 2018. https://www.w3.org/TR/activitypub/

[13] D. Watts, S. Strogatz. "Collective dynamics of 'small-world' networks." *Nature*, 393:440–442, 1998.

[14] A.-L. Barabási, R. Albert. "Emergence of Scaling in Random Networks." *Science*, 286(5439):509–512, 1999.

[15] A. D. Broido, A. Clauset. "Scale-free networks are rare." *Nature Communications*, 10:1017, 2019.

[16] R. Albert, H. Jeong, A.-L. Barabási. "Error and attack tolerance of complex networks." *Nature*, 406:378–382, 2000.

[17] R. Cohen, K. Erez, D. ben-Avraham, S. Havlin. "Resilience of the Internet to random breakdowns." *Physical Review Letters*, 85(21):4626–4628, 2000.

[18] R. Pastor-Satorras, A. Vespignani. "Epidemic spreading in scale-free networks." *Physical Review Letters*, 86(14):3200–3203, 2001.

[19] C. Castellano, R. Pastor-Satorras. "Thresholds for epidemic spreading in networks." *Physical Review Letters*, 105(21):218701, 2010.

[20] A. Rapoport. "Spread of information through a population with socio-structural bias: I. Assumption of transitivity." *Bulletin of Mathematical Biophysics*, 15(4):523–533, 1953.

[21] M. Girvan, M. Newman. "Community structure in social and biological networks." *Proceedings of the National Academy of Sciences*, 99(12):7821–7826, 2002.

[22] K. Menger. "Zur allgemeinen Kurventheorie." *Fundamenta Mathematicae*, 10:96–115, 1927.

[23] Nostr Protocol. "NIP-46: Nostr Connect." https://github.com/nostr-protocol/nips/blob/master/46.md, 2023.

[24] Permissionless Tech. "BitChat Protocol Whitepaper." https://github.com/permissionlesstech/bitchat, 2025.

[25] A. Lancichinetti, S. Fortunato, and F. Radicchi. "Benchmark graphs for testing community detection algorithms." *Physical Review E*, 78(4):046110, 2008.